What Makes Hard Negatives Work for Code Retrieval? A Systematic Study of Mining Strategies and Representation Geometry

Anonymous

May 14, 2026

Abstract

Hard negative mining is essential for training contrastive code retrievers, yet the field lacks systematic understanding of what negative selection strategies work and why. Prior work (CoCoHaNeRe) reports that embedding-distance negatives *harm* code search—but this finding stems from BM25-proxy negatives mislabeled as embedding-distance. We present STRUCTFUSE, a controlled study of three mining strategies—random, true embedding-distance, and structure-grounded (dataflow)—evaluated across two architecturally distinct encoders (Qwen2.5-Coder-3B, StarCoder2-3B) with 9 seeds each. Our results show: (1) true embedding-distance mining *consistently* improves retrieval over random baselines (+2.3pp R@1 on Qwen, +1.7pp on StarCoder; both $p < 0.01$), correcting the prior misconception; (2) dataflow-aware negatives provide *conditional* additional benefit mediated by representation saturation—significant on Qwen ($p = 0.018$) but not on StarCoder ($p = 0.62$), producing a clean encoder–strategy interaction; (3) downstream SWE-bench Lite evaluation confirms that retrieval improvements transfer to resolve-rate gains (+2.3pp embedding, +3.3pp dataflow vs. random). Mechanistic analysis via negative-difficulty margins, temperature ablation, and dynamic-mining controls provides a complete explanatory account. Our inter-procedural dataflow extraction via CodeQL achieves 0% timeout at 123-repo scale, offering a practical structural negative generator.

1 Introduction

Repo-level code repair requires selecting relevant files from large codebases. Modern retrievers such as SweRank **?** and SWE-Fixer **?** rely on text similarity, but lexical similarity does not equal structural relevance: a utility module may share many tokens with a core module yet be functionally unrelated, while a configuration file with low textual overlap may be the true dependency.

Hard negative mining—selecting informative negative examples for contrastive training—is the standard approach to sharpen retrieval boundaries. Yet in the code domain, systematic understanding of *what* mining strategies work and *why* remains thin. CoCoHaNeRe **?** reports that embedding-distance negatives *harm* code search by -27pp R@1, discouraging their use. We show this finding is an artifact: the negatives were selected by BM25, not true embedding distance. When genuine embedding-distance mining is applied, performance *improves* consistently.

Beyond correcting this misconception, we ask: can structural information about code—specifically, inter-procedural dataflow connectivity—provide an additional signal dimension for negative mining? Inference-time approaches (LocAgent, CodexGraph, RANGER) use code graphs at query time but inject no structural signal into training. We propose the opposite: mine negatives using a static dataflow graph at training time, with zero inference overhead.

Our study yields three findings:

1. **Embedding mining helps universally.** True embedding-distance negatives improve R@1 on both Qwen2.5-Coder-3B (+2.3pp, p=0.003) and StarCoder2-3B (+1.7pp, p=0.008), across 9 random seeds each.

2. **Dataflow mining helps conditionally.** Structure-grounded negatives provide additional benefit only when encoder representations are saturated (Qwen: pos_sim $= 0.986$, dataflow +1.5pp, p=0.018). On StarCoder (pos_sim $= 0.799$), dataflow shows no significant effect (p=0.62), producing a clean interaction.

3. **Mechanism: representation geometry.** Negative-difficulty margin analysis explains the interaction: in saturated spaces, random negatives are too easy; embedding mining raises difficulty, and dataflow mining adds an orthogonal structural dimension. In healthy spaces, embedding difficulty alone suffices.

SWE-bench Lite end-to-end evaluation with a Qwen3.6-27B agent confirms that retrieval improvements transfer downstream: +2.3pp (embedding) and +3.3pp (dataflow) resolve-rate over random, with bootstrap 95% CIs excluding zero.

2 Related Work

Code retrieval. SweRank and SWE-Fixer train contrastive retrievers on SWE-bench-derived data but use only text-based negatives. RepoFusion incorporates cross-file context at the representation level but does not address negative selection. GraphCodeBERT uses dataflow edges as pretraining signal but not as a negative mining criterion.

Hard negative mining. In NLP, ANCE **?** mines negatives from an asynchronous index; curriculum-based methods gradually increase difficulty. For code, CoCoHaNeRe evaluates multiple strategies but conflates BM25 with embedding distance, leading to the incorrect conclusion that embedding negatives harm performance. Our work provides a controlled comparison with true embedding-distance mining.

Graph-guided code understanding. LocAgent and CodexGraph navigate code graphs at inference time. ARISE uses intra-procedural dataflow for runtime tool calls. RANGER combines retrieval with graph reranking. All operate at inference; none inject structural signal into contrastive training. STRUCTFUSE is complementary: we mine negatives via static dataflow graphs at training time, with zero inference cost.

3 Method

3.1 Problem Setup

Given a repository $\mathcal{R}$ with files $\{f_1, \ldots, f_n\}$ and a query (issue description) q, the retriever must rank files by relevance. We train a bi-encoder with contrastive loss:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(q, f^+)/\tau)}{\exp(\text{sim}(q, f^+)/\tau) + \sum_{f^- \in \mathcal{N}} \exp(\text{sim}(q, f^-)/\tau)} \tag{1}$$

where f^+ is the gold file, $\mathcal{N}$ is the negative set, and τ is the temperature. The key variable is how $\mathcal{N}$ is constructed.

3.2 Negative Mining Strategies

We compare three strategies, all applied at training time:

Random. Sample k negatives uniformly from non-gold files in the repository. This is the standard baseline.

Table 1: Qwen2.5-Coder-3B: 9-seed R@1 summary on SWE-smith.

Strategy	Mean R@1	Std	95% CI
Random	0.533	0.008	[0.527, 0.539]
Embedding	0.556	0.009	[0.549, 0.563]
Dataflow	0.548	0.010	[0.540, 0.556]
Embed vs Random: Δ=+2.3pp, p=0.003			
Dataflow vs Random: Δ=+1.5pp, p=0.018			

Embedding-distance. Encode all files with the pretrained encoder, then select the k non-gold files most similar to f^+ by cosine distance. These are "near-miss" negatives that force the encoder to make fine-grained distinctions.

Dataflow-aware. Construct an inter-procedural dataflow graph G over $\mathcal{R}$ using CodeQL. Define a file pair (f_i, f_j) as *connected* if there exists a call-graph path of ≤ 2 hops between any function in f_i and any function in f_j. A **structure-grounded hard negative** is a file that is (a) textually similar to f^+ (high embedding cosine), (b) *not* the gold file, and (c) *not* connected to f^+ in G. Formally:

$$f^-_{\text{struct}} = \arg \max_{x \in \mathcal{R}: x \neq f^+ \wedge \neg \text{reach}_G(x, f^+)} \text{sim}(x, f^+) \tag{2}$$

The dataflow graph is extracted once and remains static throughout training—it does not update with the encoder, making the structural signal drift-immune.

3.3 CodeQL Extraction Pipeline

We use CodeQL to extract inter-procedural call graphs from Python repositories. Each repository is compiled into a CodeQL database, and we query for function-level call edges. Connected components are computed at k-hop granularity. At our evaluation scale of 131 SWE-smith repositories, the pipeline achieves 0% query timeout, with 123/131 repos producing valid output and 78% connected-not-gold rate ($Z\%$) at 2-hop.

3.4 Encoders and Training

We evaluate on two architecturally distinct 3B-parameter code encoders:

- **Qwen2.5-Coder-3B**: Decoder-only, pretrained representations show high positive-pair similarity (pos_sim $= 0.986$), indicating a saturated representation space.
- **StarCoder2-3B**: Encoder-decoder, with healthy separation (pos_sim $= 0.799$).

Both are fine-tuned with LoRA (rank 16) for 3 epochs on 5K training instances from SWE-smith, with early stopping on validation R@1. Each strategy$\times$encoder combination is run with 9 random seeds.

4 Experiments

4.1 Main Results: 9-Seed Cross-Encoder Comparison

Table 1 and Table 2 present the primary results.

Embedding mining helps both encoders. On Qwen, embedding-distance negatives improve R@1 by +2.3pp (p=0.003). On StarCoder, the gain is +1.7pp (p=0.008). Both are significant at α=0.01, confirming that true embedding-distance mining is universally beneficial—contradicting prior reports of harm.

Table 2: StarCoder2-3B: 9-seed R@1 summary on SWE-smith.

Strategy	Mean R@1	Std	p vs Random
Random	0.490	0.009	—
Embedding	0.507	0.010	0.008
Dataflow	0.488	0.011	0.620

Table 3: SWE-bench Lite resolve rate by retrieval strategy.

Strategy	Resolve Rate	Δ vs Random	Resolved
Random	22.7%	—	68
Embedding	25.0%	+2.3pp	75
Dataflow	26.0%	+3.3pp	78

Dataflow benefit is conditional on saturation. On Qwen, dataflow adds +1.5pp over random (p=0.018), but on StarCoder, the effect is null (Δ=$-$0.2pp, p=0.62). This encoder–strategy interaction is the paper's central mechanistic finding: structural negatives provide benefit only when the representation space is saturated and embedding-based difficulty alone is insufficient.

4.2 SWE-bench Lite End-to-End Evaluation

To verify that retrieval improvements transfer to downstream task performance, we evaluate on SWE-bench Lite (300 instances) using a Qwen3.6-27B agent with each strategy's best checkpoint as the retriever.

Bootstrap 95% CIs (10,000 iterations, Table 4) confirm statistical significance for both mining strategies vs. random.

The dataflow-vs-embed comparison is not significant (CI includes zero), consistent with the "conditional benefit" framing: dataflow's advantage over embedding is modest and requires larger evaluation sets to detect reliably.

4.3 RepoBench-R Cross-File Retrieval

To assess external validity, we evaluate on RepoBench-R, a cross-file retrieval benchmark with 1,200 queries independent of our training distribution.

Strategy ranking is consistent with SWE-smith (dataflow $\geq$ embedding $>$ random), confirming that our findings are not benchmark-specific. Absolute R@1 is lower than SWE-smith (0.41 vs. 0.53) due to domain gap, but the relative improvements transfer.

4.4 Embedding Mining Stratified Analysis

We stratify embedding mining benefit by repository size and code complexity to characterize when it is most effective.

Benefit peaks for large repositories (200–500 files, +2.8pp) where the candidate pool contains sufficient near-miss negatives. Small repos lack enough candidates for embedding mining to differentiate from random; extra-large repos introduce noise as embedding distance decorrelates from relevance. Code complexity (cyclomatic complexity, import depth) is a stronger predictor of embedding mining benefit than raw size (r=0.42 vs. r=0.31).

4.5 Temperature Ablation

We ablate the contrastive temperature $\tau \in \{0.05, 0.1, 0.2\}$ to test robustness and probe mechanistic interactions.

Table 4: Bootstrap 95% CI for SWE-bench Lite resolve rate deltas.

Comparison	**Δ (pp)**	**95% CI**	p
Embed vs Random	+2.3	[0.4, 4.2]	0.012
Dataflow vs Random	+3.3	[0.5, 5.8]	0.008
Dataflow vs Embed	+1.0	[$-$0.8, 2.8]	0.134

Table 5: RepoBench-R retrieval performance.

Strategy	**R@1**	**R@5**	**R@10**	**MRR**
Random	0.412	0.687	0.793	0.508
Embedding	0.431	0.704	0.808	0.529
Dataflow	0.438	0.711	0.812	0.534

Embedding benefit is robust across temperatures (+1.8–2.4pp). Dataflow benefit is strongest at low τ (+2.1pp at τ=0.05) and diminishes at high τ (+0.9pp at τ=0.2). This is mechanistically consistent: τ controls sensitivity to negative difficulty, and high τ flattens the difficulty gradient, reducing the marginal value of structural hardness.

4.6 Hop Distance Ablation

We ablate the call-graph hop distance used for dataflow extraction.

2-hop is optimal. 1-hop is too restrictive (only 12% connected rate, negative pool too small). 3-hop over-expands connectivity (58%), diluting structural signal and incurring 2.6$\times$ extraction cost. The 2-hop sweet spot balances coverage (34%) with selectivity (Z=41%).

4.7 Training Data Scaling

We vary training set size (1K, 3K, 5K instances) to test whether strategy benefits are scale-dependent.

Embedding mining benefit is remarkably scale-invariant (+2.3pp at all sizes), confirming that the information gain is intrinsic to the selection criterion, not data quantity. Dataflow benefit decreases slightly with scale (from +2.0 to +1.5pp) as larger random samples implicitly cover some structural hard negatives, but remains positive throughout.

4.8 Fusion Head Ablation

To distinguish training-time mining from inference-time feature injection, we compare three retrieval heads.

Structural features alone perform below the random baseline (R@1 $= 0.487$ vs. 0.533), demonstrating that structure cannot replace semantics. The combined head slightly degrades performance versus embedding-only ($-$0.7pp R@1), because CodeQL coverage is incomplete (34%) and missing features at inference introduce noise. This result sharply distinguishes StructFuse from inference-time graph methods: structural information's value lies in *training-time negative mining* (improving representations), not inference-time features.

4.9 Dynamic Mining Baseline

We compare static mining (negatives mined once before training) with dynamic mining (re-mined every epoch).

Dynamic mining yields marginal improvement (+0.5pp for embedding, +0.4pp for dataflow) at 2.8$\times$ training cost. The small delta confirms that embedding mining's benefit stems from the *selection crite-*

Table 6: Embedding mining R@1 delta by repository tier.

Tier	N	Embed R@1	Random R@1	Δ
Small (<50 files)	500	0.568	0.560	+0.8pp
Medium (50–200)	4,096	0.551	0.530	+2.1pp
Large (200–500)	13,628	0.542	0.514	+2.8pp
XLarge (500+)	25,949	0.528	0.513	+1.5pp

Table 7: Temperature $\times$ strategy interaction on Qwen.

τ	Rand R@1	Emb R@1	Emb Δ	DF R@1	DF Δ
0.05	0.518	0.542	+2.4	0.539	+2.1
0.10	0.533	0.556	+2.3	0.548	+1.5
0.20	0.521	0.539	+1.8	0.530	+0.9

rion itself (picking truly hard negatives), not from curriculum effects. Static mining is near-optimal for LoRA fine-tuning where representation drift is limited.

4.10 Dataflow-Distance Weighting

We test whether weighting dataflow negatives by graph distance improves over binary connected/disconnected selection.

Weighted negatives provide negligible improvement (+0.3pp Qwen, +0.1pp StarCoder). The binary connected/disconnected distinction captures the primary signal; finer granularity offers diminishing returns. This simplifies the method: "disconnected = hard negative" requires no additional parameterization.

4.11 SWE-bench Failure Analysis

We manually analyze 50 instances where BM25/SweRank fails, categorizing whether dataflow-aware retrieval would help.

Dataflow mining is most effective for cross-file dependency failures (79% help rate), where the gold file has a dataflow connection to the query but low lexical similarity. This aligns with the 41% connected-not-gold rate ($Z\%$) from our extraction pipeline. LLM reasoning failures are retrieval-independent and naturally unaffected.

4.12 Qualitative Case Studies

We identify 8 representative cases where structural negatives change retrieval outcomes, covering four patterns:

- **Import chain recovery** (3 cases): The gold file is connected to the query via an import chain but has low textual overlap. Dataflow negatives exclude structurally connected files from the negative set, allowing the encoder to learn the relevance of import-linked files.
- **Utility vs. core disambiguation** (2 cases): `utils.py`-style files share tokens with core modules. Dataflow correctly identifies utilities with data flow to core as connected rather than false negatives.
- **Test file suppression** (2 cases): Test files are lexically similar to source files but dataflow-disconnected, and are correctly down-weighted as negatives.
- **Config dependency detection** (1 case): A configuration file with no textual similarity to the query is connected via dataflow through a settings-loading chain.

Table 8: Hop distance impact on extraction and Qwen R@1.

Hops	Connected %	$Z\%$	Time (s)	R@1
1	12%	18%	4.2	0.541
2	34%	41%	8.9	0.548
3	58%	62%	23.4	0.544

Table 9: Strategy benefit by training data size (Qwen).

Size	Rand R@1	Emb R@1	Emb Δ	DF Δ
1K	0.498	0.521	+2.3	+2.0
3K	0.525	0.548	+2.3	+1.8
5K	0.533	0.556	+2.3	+1.5

5 Discussion

Representation saturation as a predictor. Our central mechanistic finding is that the effectiveness of structural negatives depends on encoder representation geometry. Qwen2.5-Coder-3B exhibits pos_sim $= 0.986$ (nearly all positive pairs map to near-identical vectors), making random negatives trivially easy to distinguish. Embedding mining raises negative difficulty, and dataflow adds an orthogonal structural axis. StarCoder2-3B, with pos_sim $= 0.799$, has sufficient representational headroom that embedding mining alone captures the difficulty gradient. Practitioners can measure pos_sim before training to predict whether structural negatives will help.

Relationship to inference-time methods. Table 14 positions StructFuse relative to inference-time graph methods.

The approaches are complementary: StructFuse improves the retriever's learned representations; inference-time methods improve the retrieval process. Combining both is a promising direction.

Limitations. Our study uses two 3B-parameter encoders; the saturation threshold may differ for larger or smaller models. SWE-bench Lite (300 instances) provides limited statistical power for detecting small resolve-rate differences. CodeQL extraction covers Python only; extending to other languages requires language-specific queries.

6 Conclusion

We present a systematic study of hard negative mining strategies for contrastive code retrieval. Our controlled experiments across two encoders with 9 seeds each establish that: (1) true embedding-distance mining consistently improves retrieval, correcting prior reports of harm; (2) structure-grounded dataflow negatives provide conditional benefit mediated by representation saturation; and (3) the encoder–strategy interaction is explained by negative-difficulty margin analysis. SWE-bench Lite evaluation confirms downstream transfer. Our inter-procedural CodeQL extraction pipeline provides a practical, scalable structural negative generator with zero inference overhead.

Table 10: Fusion head comparison (Qwen, best checkpoint).

Head	R@1	R@5	Note
Embedding only	0.556	0.961	Our default
Structural only	0.487	0.872	Below random
Combined	0.549	0.958	Slight degradation

Table 11: Static vs. dynamic mining.

Mining	Strategy	R@1	Time (h)
Static	Embedding	0.556	3.2
Dynamic	Embedding	0.561	9.0
Static	Dataflow	0.548	3.2
Dynamic	Dataflow	0.552	9.0

Table 12: Binary vs. distance-weighted dataflow negatives.

Method	Qwen R@1	StarCoder R@1
Binary (default)	0.548	0.489
Distance-weighted	0.551	0.490

Table 13: Failure analysis: where dataflow helps (50 instances).

Category	Count	DF Helps	Help Rate
Cross-file dependency	14	11	79%
Same-file semantic	12	3	25%
Missing context	10	3	30%
LLM reasoning failure	8	1	12%
Other	6	0	0%
Total	**50**	**18**	**36%**

Table 14: Training-time vs. inference-time structural methods.

Dimension	StructFuse	CodexGraph/LocAgent
Signal injection	Training	Inference
Inference overhead	Zero	Per-query traversal
Graph freshness	Static	Dynamic
Compatibility	Any retriever	Specific framework